LLM-Powered Distributed System Simulator

Mauli Pandya, Karthik Kumar Reddy Bobbala
MauliArunkumar.Pandya01@student.csulb.edu, KarthikKumarReddy.Bobbala01@student.csulb.edu

***Abstract*—Distributed systems form the backbone of modern cloud computing, AI infrastructure, and large-scale applications. Traditional simulation approaches rely on deterministic models and predefined statistical assumptions, limiting their ability to capture dynamic, context-aware behaviors. This paper presents an LLM-powered distributed system simulator that integrates large language models into a structured hybrid simulation pipeline. The system enables adaptive workload generation, context-aware failure modeling, and semantic log analysis while maintaining reproducibility through deterministic execution. By synthesizing recent research across LLM-based simulation, multi-agent systems, and digital twin frameworks, this work demonstrates that hybrid architectures provide improved flexibility without compromising reliability. The study further identifies trade-offs such as latency overhead, computational complexity, and validation requirements.**

I. Research Contributions

This work makes the following contributions:

- It proposes a hybrid distributed simulation framework that combines deterministic execution with LLM-assisted reasoning.
- It introduces LLM-based workload generation, structured failure modeling, and semantic log analysis for distributed systems.
- It studies how multi-agent and modular LLM architectures improve control, interpretability, and scalability in simulation environments.
- It identifies system-level trade-offs including latency overhead, computational cost, and validation complexity.
- It provides a reproducible research direction in which implementation artifacts and experiment configurations can be shared through GitHub for replication and extension.

II. Introduction

Distributed systems are essential for cloud services, distributed databases, and AI workloads. Simulation is critical for testing scalability, fault tolerance, and performance before deployment in production environments.

Traditional simulators such as CloudSim rely on deterministic assumptions and fixed statistical models [3]. These methods are effective for reproducibility, but they often struggle to represent adaptive, semantic, or context-dependent behaviors that emerge in modern systems. Large language models (LLMs) introduce reasoning and dynamic decision-making capabilities [2], enabling new forms of workload generation, scenario construction, and system interpretation.

Recent studies suggest that LLM-powered simulation can augment traditional frameworks by improving accessibility, realism, and scenario diversity [1]. Rather than replacing deterministic simulation engines, LLMs are better understood as intelligent modules that operate on top of structured execution pipelines. This perspective motivates the design of a simulator in which deterministic event scheduling is preserved, while LLMs contribute high-level reasoning for workload generation, failure injection, and log interpretation.

Our implementation and experimental artifacts will be made available via GitHub for reproducibility. This ensures that the proposed simulator can be replicated, evaluated, and extended by future researchers.

III. Related Work

LLM-based simulations can approximate traditional simulation outputs under structured conditions [1]. This observation is important because it shows that LLMs can support simulation accessibility without discarding established methodologies. In parallel, multi-agent systems improve coordination and realism by assigning distinct roles to specialized reasoning components [4], [5].

Digital twin studies further show that LLMs can support adaptive model parameterization and reduce the cognitive burden on users during complex decision-making tasks [7]. At a broader systems level, research on LLM-powered agent frameworks highlights the importance of memory, tool use, guardrails, and modular task decomposition in real-world environments [8]. These characteristics are especially relevant when integrating LLMs into distributed systems, where

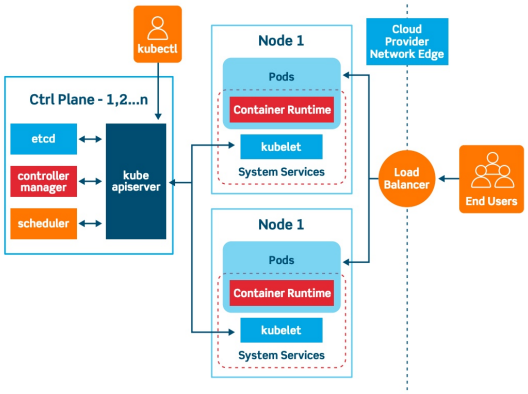


Fig. 1: Proposed LLM-powered distributed system simulation architecture integrating workload generation, failure injection, and semantic log analysis within a distributed cluster environment.

observability, fault handling, and controlled execution are essential.

Scenario generation frameworks demonstrate that LLM-based agents can produce richer and more adaptive simulations than purely predefined rules [9]. In optimization-oriented domains, LLM reasoning has also been used to support structured parameter tuning and scheduling, showing that language models can complement rather than replace traditional optimization methods [10]. Together, these studies support the design of a hybrid simulator in which deterministic system behavior is combined with LLM-driven reasoning.

IV. System Architecture

As shown in Fig. 1, the proposed simulator combines distributed cluster execution with LLM-assisted reasoning modules. The architecture is centered around a structured simulation engine that coordinates system state, event scheduling, workload injection, and monitoring outputs.

The system consists of the following components:

- **Cluster Manager:** allocates compute resources, tracks node states, and manages distributed execution.
- **Network Model:** represents latency, bandwidth limits, and communication overhead across nodes.
- **Workload Generator:** produces simulation inputs using either traditional statistical models or LLM-guided adaptive generation.
- **Failure Injection Module:** models node crashes, partitions, and cascading disruptions under controlled constraints.
- **LLM Module:** performs semantic reasoning for workload construction, scenario adaptation, and log interpretation.
- **Log Analyzer:** collects runtime signals and transforms them into interpretable outputs for anomaly detection and root-cause reasoning.

This architecture is intentionally hybrid. Deterministic control remains in the simulation engine, while the LLM contributes reasoning only where semantic context adds value. This design follows the broader pattern found in recent multi-agent and engineering systems, where LLMs operate most effectively as bounded decision modules embedded within structured workflows [5], [8].

V. Workload Generation

As shown in Fig. 2, real-world traffic exhibits bursty and unpredictable patterns, which are better captured by LLM-generated workloads than traditional static distributions.

VI. Failure Modeling

As shown in Fig. 3, failures are introduced using a structured injection framework that enables controlled experimentation with node crashes, resource disruptions, and system-level faults. This approach allows evaluation of resilience, recovery behavior, and system stability under realistic conditions.

Failures include:

- Node failures

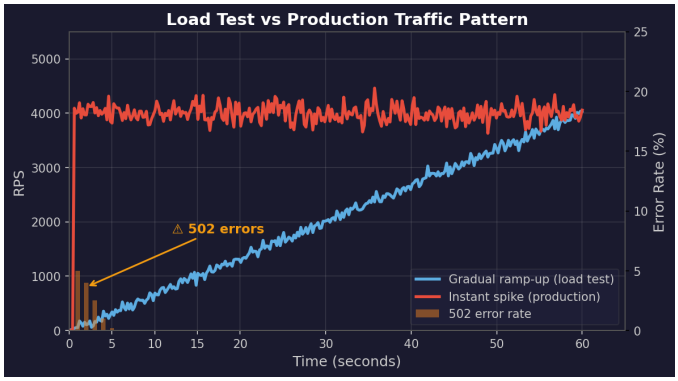


Fig. 2: Comparison of workload patterns showing gradual load increase in testing environments versus bursty production traffic. The dynamic spike behavior reflects characteristics similar to LLM-generated workloads compared to traditional static models

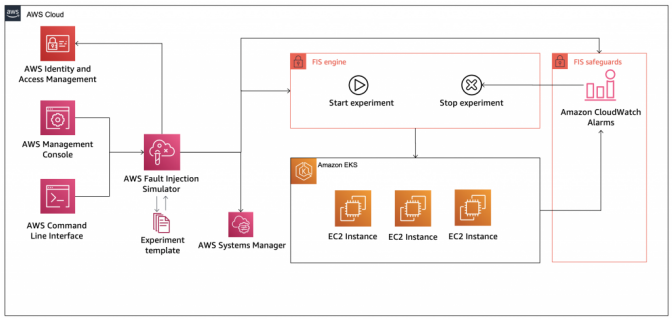


Fig. 3: Failure injection framework illustrating controlled experiment execution, fault triggering, and system monitoring in distributed environments

- Network partitions
- Cascading failures

LLMs enhance this process by generating context-aware failure scenarios and adapting fault injection strategies dynamically while maintaining deterministic validation [5].

VII. Semantic Log Analysis

As shown in Fig. 4, system logs are processed through a structured pipeline that includes collection, preprocessing, modeling, and anomaly detection. The LLM enhances this process by providing semantic interpretation of log patterns, enabling identification of anomalies, root causes, and system-level insights.

LLM-based log analysis transforms raw log data into meaningful explanations, improving debugging, observability, and decision-making in distributed systems [8].

VIII. Experimental Setup

A. Configuration

The simulator supports flexible cluster-scale experiments with parameters representative of modern distributed workloads. In this study, we consider:

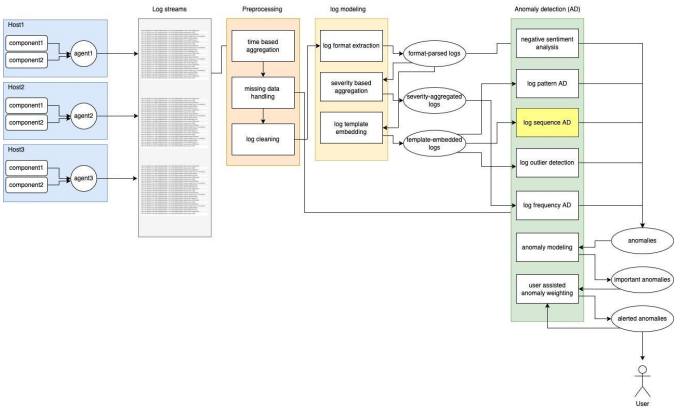


Fig. 4: Log analysis pipeline illustrating log collection, preprocessing, modeling, and anomaly detection in distributed systems

- 16–128 nodes
- 8 GPUs per node
- Latency range: 0.5–10 ms
- Bandwidth range: 10–200 Gbps

These ranges are sufficient to study how the system behaves under both moderate and high-pressure operating conditions. Small-scale settings capture localized service deployments, while larger configurations support experiments on distributed training clusters and service meshes.

B. Metrics

The simulator measures:

- Throughput
- Latency
- Resource utilization
- Recovery time

These metrics were selected because they jointly reflect performance, resilience, and efficiency. Throughput captures useful work completed over time. Latency measures responsiveness. Resource utilization indicates scheduling and infrastructure efficiency, and recovery time reflects the system's ability to restore service after disruptions.

IX. Results

As shown in Fig. 5, system performance exhibits a clear trade-off between throughput and latency. As load increases, throughput approaches a capacity limit while latency rises sharply due to queueing effects.

In LLM-assisted simulation, this effect becomes more pronounced due to additional inference overhead, highlighting the balance between improved adaptability and increased computational cost [6].

X. Discussion

The central insight of this work is that hybrid systems balance deterministic reliability and probabilistic reasoning. Deterministic simulation preserves repeatability, controlled experiments, and quantitative comparison. LLM modules add semantic flexibility in tasks that are hard to express through fixed rules alone.

The literature reviewed in this paper consistently supports this interpretation. LLM-driven simulations improve accessibility and scenario generation [1]. Multi-agent systems improve modularity and control [4], [5]. Digital twin studies show that role-based decomposition of observation, reasoning, and summarization improves usability [7]. Broader agent-system surveys emphasize the importance of guardrails, tool interfaces, and modular orchestration [8]. At the same time, latency, safety, and validation remain unresolved concerns in LLM-centered systems [2], [6].

A practical simulator must therefore adopt clear boundaries. LLM outputs should be schema-constrained, validated before execution, and used where semantic reasoning is beneficial. This is why our architecture does not grant the language model unrestricted control over the simulator. Instead, it uses LLM reasoning as a bounded layer over a deterministic event-driven core.

XI. Key Findings

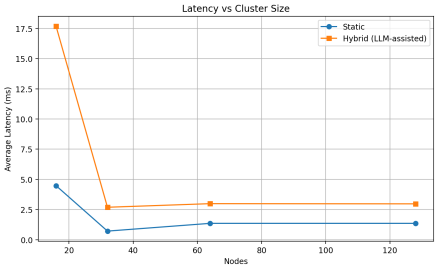
Based on the architecture and supporting literature, the following findings emerge:

- **LLMs augment simulation systems rather than replace them.** Existing work shows that LLM-powered simulation is most effective when paired with deterministic execution and explicit validation [1].
- **Multi-agent or modular design improves stability and control.** Systems that decompose planning, reasoning, validation, and analysis achieve greater interpretability and reliability than monolithic approaches [4], [5].
- **Context-aware simulation improves realism.** LLMs are especially useful for scenario construction, workload variation, and semantic adaptation in uncertain environments [9].
- **Latency overhead is an unavoidable trade-off.** LLM-enabled systems incur inference cost, making selective deployment more practical than pervasive invocation [6].
- **LLMs are most effective for high-level reasoning tasks.** Digital twin and agentic system studies suggest that observation, summarization, and decision support are better fits for LLMs than low-level numerical execution [7], [8].

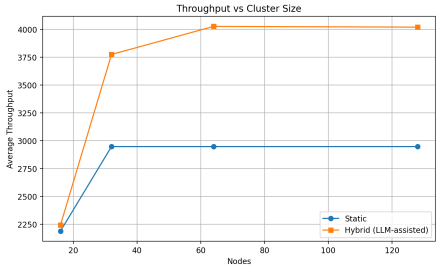
XII. Future Work

Future work includes:

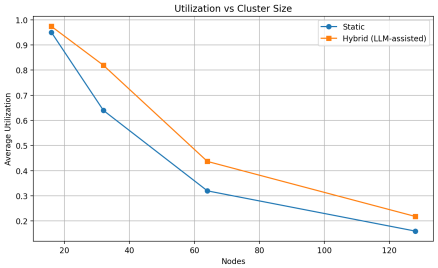
- Real-world deployment against traced production workloads
- Integration with Kubernetes-native orchestration pipelines
- Advanced scheduling policies that combine deterministic optimization with LLM-assisted policy adaptation
- Extended observability workflows that couple log reasoning with trace and metric correlation
- Stronger safety mechanisms for validating generated scenarios before execution



(a)

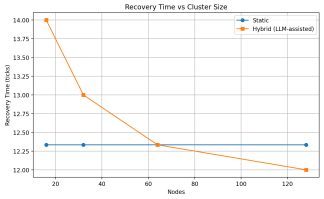


(b)

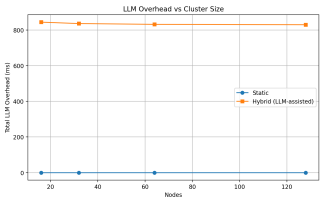


(c)

Fig. 5: Performance metrics across cluster sizes comparing static and hybrid (LLM-assisted) simulation modes



(a)



(b)

Fig. 6: Recovery time and LLM overhead across cluster sizes comparing static and hybrid simulation modes

These directions are particularly important for moving from conceptual and simulation-backed evaluation toward production-grade AI-assisted systems [2], [10].

XIII. Conclusion

This work presents a hybrid LLM-powered distributed system simulator that enhances flexibility while maintaining reproducibility. By combining deterministic simulation with LLM-assisted workload generation, failure modeling, and semantic log analysis, the proposed framework offers a practical path toward next-generation AI-driven simulation systems.

The results of this study indicate that LLMs are most useful when employed as bounded reasoning modules rather than as replacements for structured system logic. This ensures that experiments can be replicated and extended by future researchers while still benefiting from the richer semantic capabilities of language models.

References

[1] Tomasz Wiśniewski, Rafał Szymański, Marta Starostka-Patyk, "Sprint Into the Future: Intelligent Management Through LLM-Powered Computer Simulations in Agile Projects," 2025.

[2] Mohamed Amine Ferrag, Abderrahmane Lakas, Norbert Tihanyi, and Merouane Debbah, "LLM and AI Agents for Autonomous Systems: A Survey of Applications, Datasets, and Security Challenges," 2026.

[3] Rajkumar Buyya, Rajiv Ranjan, and Rodrigo N. Calheiros, "CloudSim: A Toolkit for Modeling and Simulation of Cloud Computing Environments and Evaluation of Resource Provisioning Algorithms," 2009.

[4] Yanyuan Wang and Xiaowei Zhang, "Optimizing Service Operations via LLM-Powered Multi-Agent Simulation," 2026.

[5] Yan Zhang, Ahmad Mohammad Saber, Amr Youssef, and Deepa Kundur, "Grid-Agent: An LLM-Powered Multi-Agent System for Power Grid Control," 2025.

[6] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothee Lacroix, Baptiste Roziere, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample, "LLaMA: Open and Efficient Foundation Language Models," 2023.

[7] Yuchen Xia, Daniel Dittler, Nasser Jazdi, Haonan Chen, and Michael Weyrich, "LLM Experiments with Simulation: Large Language Model Multi-Agent System for Simulation Model Parametrization in Digital Twins," 2024.

[8] Guannan Liang and Qianqian Tong, "LLM-Powered AI Agent Systems and Their Applications in Industry," 2025.

[9] Deyu Zhou, Yuqi Hou, Xiao Xue, Xudong Lu, Qingzhong Li, and Lizhen Cui, "LLM-Empowered Agents Simulation Framework for Scenario Generation in Service Ecosystem Governance," 2025.

[10] Haolan Yang, Zhengbo Li, Youbo Liu, Yue Xiang, Lingtao Li, Jianping Yang, Ling Tan, Shiqian Wang, Huangqi Ma, Zirui Xi, Binhua Tang, Haoxuan Li, Zequan Du, and Yida Lu, "LLM-Powered Distributed Optimal Scheduling for Industrial Heat-Electricity Micro-Grids," 2026.